



ORACLE®

MySQL
Replication and Scalability

Tony Darnell
Principal Sales Consultant

ORACLE



Disclaimer

- The following is intended to outline our general product direction.
- It is intended for information purposes only, and may not be incorporated into any contract.
- It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions.
- The development, release, and timing of any features or functionality described for Oracle's products remains at the sole discretion of Oracle.



Expectations

- Will look at the different ways to utilize replication
- Will not go into detail how to setup replication by editing files, setting parameters, etc.
- Developer versus DBA approach

ORACLE

This presentation is a bit different in that we are usually talking to DBA's about MySQL.

Since this is a developer's conference, we are going to be looking at replication from a developer's point of view.

So, we aren't going to spend a lot of time on how to configure replication.

But we are going to cover the basic uses for replication, so that as you design applications or systems, you will have a little bit of knowledge on how you could implement replication.

What is Replication?

- Replication enables data from one MySQL database server (master) to be replicated to one or more MySQL database servers (slaves).



ORACLE

So, what is replication?

Replication enables data from one MySQL database server (called the master) to be replicated or duplicated to one or more MySQL database servers (the slaves).

Replication is controlled through a number of different options and variables, which controls the core operations of replication, and the databases and filters that can be applied to your data.

You can use replication to solve a number of different problems, including problems with performance, supporting the backup of different databases, and as part of a larger solution to possibly remedy system failures.

The master server writes all database changes to the binary log – or binlog. The slave checks the binlog for these changes and writes them into a relay log. The relay log then writes these changes to the database.

Replication Types & Formats

- Types:
 - Asynchronous - slaves do not need to be connected permanently to receive updates from the master
 - Semi-synchronous - at least one slave acknowledges receipt from master
 - Synchronous - slaves must acknowledge receipt
- Formats:
 - Statement-based - propagation of SQL statements from master to slave
 - Row-based - changes in individual rows
 - Mixed - combination of statement and row-based

ORACLE

There are three types of replication – and when we say types, we are talking about how the data transfer is managed when transferred from the master to the slaves.

MySQL Replication is asynchronous by default – slaves do not need to be connected permanently to receive updates from the master. This means that updates can occur over long-distance connections and even over temporary or intermittent connections such as a dial-up service. Depending on the configuration, you can replicate all databases, selected databases, or even selected tables within a database.

In MySQL 5.5, semi-synchronous replication is supported in addition to the default asynchronous replication. With semi-synchronous replication, a commit performed on the master side is held until at least one slave acknowledges that it has received and logged the events for the transaction.

In synchronous, the slaves must acknowledge receipt from the master - similar to how MySQL Cluster works, and you will hear more about this in the Cluster presentation.

Statement-based replication is based on the simple propagation of SQL statements from a master to slave.

In row-based replication, binary logging records changes in individual table rows. The master writes events to the binary log that indicate how individual table rows are changed.

When the mixed format is in effect, statement-based logging is used by default, but automatically switches to row-based logging in particular cases when it is less costly.

Replication using the mixed format is often referred to as mixed-based replication or mixed-format replication.

And when using MIXED format, the binary logging format is determined in part by the storage engine being used and the statement being executed.



Replication Disadvantages

- Not true High Availability - data can be lost on system failure
- Complex fail-over/fail-back with more than one slave
- A recovered master may have non-binlogged changes.
- Slaves can lag behind the master

ORACLE

Replication is not a true high availability solution. Data can and probably will be lost on a system failure.

If the master does fail, fail-over and fail-back is fairly complex – especially if you have more than one slave.

But, if you do implement replication, it is a good idea to have a well-thought out disaster recovery plan – and test it if possible.

If the master fails and there are changes that were not written to the binlog and not retrieved by the slave, then there will be lost data.

The slave can lag behind the master depending upon the load of the master server, network inefficiencies and how often the slave is retrieving data from the master.

Even if you have a relatively small write load such as 1,000 writes per second, if the slave is five seconds behind the master, and the master fails, then you could miss or lose several thousand changes.

Uses of Replication

- High Availability (Failover)
- Scalability – scale up/scale out
- Data Security/Backups
- Analytics
- Long-distance Data Distribution

ORACLE

Here are some of the common uses for replication.

High Availability – replication isn't a true High Availability solution like cluster, as data may and probably will be lost on a system failure – but it does allow you to fail over to a standby server if and when the master fails.

Scalability – As your system grows, you can handle an increase in two ways – scale up or scale out. Scaling up means buying a larger and more powerful server to handle the increased load. Scaling out means to add more servers to handle the increased load.

Of the two, scaling out is the more popular solution because it typically involves buying a batch of low-cost servers and it is more cost-effective.

And, with scale-out solutions, you are spreading the load among multiple slaves to improve performance. In this environment, all writes and updates take place on the master server. Reads, however, may take place on one or more slaves. So, this model can improve the performance of writes (since the master is dedicated to only performing updates), while dramatically increasing read speeds across an increasing number of slaves.

Data security - because data is replicated to the slave, and the slave can pause the replication process, it is possible to run backup services on the slave without corrupting the corresponding master data.

Analytics - live data can be created on the master, while the analysis of the information can take place on the slave without affecting the performance of the master.

Long-distance data distribution - if a branch office would like to work with a copy of your main data, you can use replication to create a local copy of the data for their use without requiring permanent access to the master.

Replication Configuration

- Based upon the binary logging mechanism
- Master is “dumb” in this scenario
- Each slave keeps binary log coordinates
- Slaves can be connected/disconnected at will
- Master and slaves must have unique id
- Slaves must know master host id, log file name, and bin log position.

ORACLE

As we mentioned earlier, replication between servers in MySQL is based on the binary logging mechanism. The MySQL instance operating as the master (which is the source of the database changes) writes updates and changes as “events” to the binary log. The information in the binary log is stored in different logging formats according to the database changes being recorded. Slaves are configured to read the binary log from the master and to execute the events in the binary log on the slave’s local database.

In this scenario, the master is “dumb”. Once the binary logging has been enabled, all statements are recorded in the binary log. Each slave then receives a copy of the entire contents of the binary log.

The slave to decide which statements in the binary log should be executed; the master logs all events. If you do not specify otherwise, all events in the master binary log are also executed on the slave. If required, you can configure the slave to process only events that apply to particular databases or tables.

So, each slave keeps a record of the binary log coordinates: The coordinates are the file name and position within the binary log file that the slave has read and processed from the master. This means that multiple slaves can be connected to the same master and executing different parts of the same binary log.

Because the slaves control this process, individual slaves can be connected and disconnected from the master server without affecting the master’s operation.

Also, since each slave remembers it’s own position within the binary log, it is possible for slaves to be disconnected, reconnected and then they will “catch up” to the master by continuing from a recorded position in the binlog.

Both the master and each slave must be configured with a unique ID (using the server-id option). In addition, each slave must be configured with information about the master host name, log file name, and position within that file.

MySQL Binary Log (binlog)

- Binlog – a record of all changes made to the database.
 - Comprised of multiple files
 - Old binlog files may be removed
 - Binlog Index file – tracks what binlog files exist
- Binlog is used for:
 - Replication
 - Point-in-time recovery
 - Auditing (limited use)

ORACLE

Again, replication is possible because of the binary log – or binlog.

We will need to understand how the binlog works in order to have control over the replication process and in order to be able to fix any problems that occur.

The purpose of the binlog is to record changes made to the tables in the database. So the binlog does not record any queries that do not change data.

The binary log contains “events” that describe these database changes such as table creation operations or changes to table data. It also contains events for statements that potentially could have made changes (for example, a DELETE statement which matched zero rows) – that is, unless row-based logging is used.

And the binary log also contains information about the execution time for each statement that updated data.

The binlog is not just a single file, but a set of files that allows for easier database management – so you can remove old logs without disturbing newer ones.

There is also a binlog index file, which keeps track of which binlog files exist. Only one binlog file is the active file – and this active file is the one that is currently being used for data writes.

The binlog can then be used for replication, for point-in-time recovery in backups, and in some limited cases for auditing of data.

Binlog Example

```
mysql> create table test (text TEXT);
Query OK, 0 rows affected (0.56 sec)

mysql> insert into test values ("Replication!");
Query OK, 1 row affected (0.46 sec)

mysql> select * from test;
+-----+
| text          |
+-----+
| Replication!  |
+-----+
1 row in set (0.00 sec)
```

ORACLE

Let's take a look at an example of writing to the binlog.

In the first step, we are going to create a table named test consisting of a single text column named TEXT.

We will insert into this table a text value – “Replication!”.

And then we will do a select statement to select all rows from test, and we retrieve one row.

Binlog Events

```
mysql> show binlog events\G
***** 2. row *****
  Log_name: mysql-bin.000001
    Pos: 107
  Event_type: Query
  Server_id: 1
End_log_pos: 210
  Info: use `sample`; create table test (text TEXT)
***** 4. row *****
  Log_name: mysql-bin.000001
    Pos: 289
  Event_type: Query
  Server_id: 1
End_log_pos: 408
  Info: use `sample`; insert into test values ("Replication!")
```

ORACLE

Now we can take a look at the binlog with the “show binlog events” statement.

On this slide, we won’t see all of the binlog events – as they wouldn’t fit on the slide, so we are just looking at the binlog for the two SQL statements that we just executed. Rows 1 and 3 are not displayed.

Under row TWO, here we see the binlog entry for when we created the table named “test”.

Under row FOUR, we see where we inserted a value into the test table.

You will notice that I didn’t specify the table to be used in the earlier SQL statements – but under the info column, it shows “use sample” – as that was the database that I was using earlier.

The columns for each row are:

Log Name – the binlog file that is being referenced or was used for this statement.

Pos – this is the position in the file where the event starts – the first byte of the event

The position is key in using the binlog to replicate data and when promoting slaves to masters.

Event Type – this is the type of event – there are about 27 different event types – such as Format_desc, Stop, Query, Xid, User var, Table_map, Update_rows, Rotate, Intvar

Server ID – the id of the server that created the event

End log position – the ending byte of the event – where this event ends and where the next one begins

Info – information about the event – different information is printed for different events, but you can at least count on the query event to print the statement that it contains – unless you are using row-based replication.

Replication Setup

- Enable binary logging on master – edit my.cnf file: (under [mysql])
 - Add: log-bin = master-bin
 - Add: log-bin-index = master-bin.index
- Configure unique server ID
 - Add: server-id = 1
- Create separate user to authenticate with master (optional)

ORACLE

There are some generic tasks that are common to all replication setups:

On the master, you must enable binary logging, give a naming convention for your binlog and binlog index files and configure a unique server ID.

Edit my.cnf file – under [mysql] add "log-bin = master-bin" – and you can give it a binlog file prefix name – such as master-bin.index

If you just add "log-bin" to the my.cnf file, MySQL will create the binlog name by using the computer name or by using "mysql".

You need to give the master a server id – so add "server-id = 1".

You may want to create a separate user that will be used by your slaves to authenticate with the master to read the binary log for replication. The step is optional.

This will probably require a server restart on the master.

Cloning the Master

- Flush the tables with read lock
- Find current binlog position
- Create a backup of the master
- Unlock tables on master
- Restore the backup to slave
- Configure slave
- Start slave
- Slave will start replicating and catch-up

Master



Slave

ORACLE

Here are the steps to cloning the master – we will take a quick look at the SQL statements on the next slide.

To clone the master, since the master is running and it probably has a lot of tables in the cache, you need to flush the tables and lock the database to prevent changes before you check the last binlog position of the master.

Once the database is locked, you are ready to create a backup and you must note the binlog position. This position in the binlog is the last change that was made to the database.

Since we have locked the database, no changes are occurring on the master, the show master status command will reveal the current file and position in the binary log.

Create a backup of the master – for example, by using mysqldump.

Unlock the tables on the master, so you can allow it to continue processing queries.

Restore the backup on the slave

Recalling the last binlog position of the master that you noted, after you stopped the master server and after you created a backup, you can configure the slave and start the slave.

The slave will now catch up to the master – and it will start replicating AFTER the last binlog position – which should also be the last transaction that was committed on the master before you started the backup.

And depending upon how much data was written/changed since the backup, it could take a while for the slave to catch up to the master.

Cloning the Master

```
master> flush tables with read lock;

master> show master status\G
***** 1. row *****
      File: mysql-bin.000001
      Position: 47710
      Binlog_Do_DB:
      Binlog_Ignore_DB:

master $ mysqldump --all-databases -host=master-1 > backup.sql

master> unlock tables;

slave $ mysql --host=slave-1 < backup.sql

slave> change master to
-> MASTER_HOST = 'master-1',
-> MASTER_PORT = 3306,
-> MASTER_USER = 'slave'.
-> MASTER_PASSWORD = 'password_value',
-> MASTER_LOG_FILE = 'mysql-bin.000001'
-> MASTER_LOG_POS = 47710;

slave> start slave;
```

ORACLE

And here are the SQL steps to clone the master:

```
master> flush tables with read lock;

master> show master status\G
***** 1. row *****
      File: mysql-bin.000001
      Position: 47710
      Binlog_Do_DB:
      Binlog_Ignore_DB:

Master $ mysqldump --all-databases -host=master-1 > backup.sql

master> unlock tables;

slave $ mysql --host=slave-1 < backup.sql

slave> change master to
-> MASTER_HOST = 'master-1',
-> MASTER_PORT = 3306,
-> MASTER_USER = 'slave'.
-> MASTER_PASSWORD = 'password_value',
-> MASTER_LOG_FILE = 'mysql-bin.000001'
-> MASTER_LOG_POS = 47710;
```

You will notice that the MASTER LOG_POS(ition) is 47710 - this was taken from the SHOW MASTER STATUS up above.

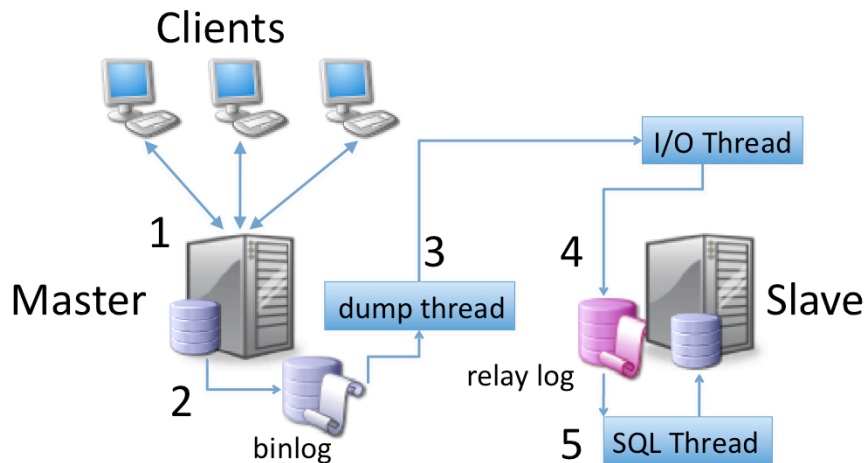
Since you have restored the database from a backup, you should have all of the database changes from the binlog up to this point.

```
slave> start slave;
```

Depending upon how long it took to restore the backup, and if the master has had a lot of activity since the backup, it might take the slave a while to catch up to the master.

Once you have a slave connected to the master, you can then use that slave to create new slaves - without having to stop the master again.

Replication Architecture Basics



Now that we have a slave configured, this is how the events flow through a replication system from the master to the slaves:

- 1 – A session on the master accepts a statement from the client, executes the statement, and synchronizes with other sessions to ensure that each transaction is executed without conflicting with other changes made by other sessions.
- 2 – Just before the statement finishes execution, an entry consisting of one or more events is written to the binary log on the master.
- 3 – After the events have been written, a dump thread is created on the master when a slave I/O thread connects, the dump thread reads the events from the binary log, and sends them over to the slaves I/O thread. There is one dump thread per connected slave.
- 4 – When the slave I/O thread receives the event, it writes it to the end of the slave's relay log.
- 5 – Once in the relay log, a slave SQL thread reads the event from the relay log and executes the event to apply the changes to the database on the slave. It is also responsible for coordinating with other MySQL threads to ensure changes do not interfere with other activities going on in the MySQL server.



Common Replication/Scale-Out Uses

- Load balancing for reads
- Load balancing for writes
 - Distributed based on data role
 - Partitioned by geographical region
- Disaster avoidance via hot standby
- Disaster avoidance through remote replication
- Backups
- Report Generation
- Filtering or partitioning data

ORACLE

Now that we have a slave up and running, let's take a look at how we can use replication for scaling out.

Load balancing for reads – since the master is occupied with updating data, it can be wise to have separate servers to answer queries. Since queries only need to read data, you can use replication to send changes on the master to the slaves – so they have current data and can process queries.

Load balancing for writes – high-traffic deployments distribute processing over many computers, sometimes several thousand. Replication plays a critical role in distributing the information to be processed. The information can be distributed in many different ways based on the business use and nature of your data.

- Distributed of data should be based on the information's role. Keep rarely updated tables on a single server, while frequently updated tables are partitioned over several servers
- Partition the data by geographic region so traffic may be directed to the closest server

Disaster avoidance via hot standby – If the master goes down, everything will stop. The easiest solution is to configure a slave with the purpose of acting as a hot standby, ready to take over the job of the master if it fails.

Disaster avoidance through remote replication – every deployment runs the risk of having a data center go down due to a disaster. To mitigate this, you may use replication to transport information between geographically remote sites.

Making backups – keeping an extra server around for making backups is very common. This extra server allows you to make your backups without having to disturb the master at all, since you can take the backup server offline and do whatever you like with it.

Report generation – creating reports from data on an active server will degrade the server's performance. If you are running a lot of reports, it's worth creating a slave for just this purpose.

Filtering or partitioning data – if the network connection is slow, or if some data should not be made available to certain clients, you can add a server to handle data filtering.

Scaling Out Reads, Not Writes

- Each slave handles same write load as master

$$\text{Avg. Load} = \frac{\sum \text{Read Load} + \sum \text{Write Load}}{\sum \text{Capacity}}$$

(master)

$$\text{Avg. Load} = \frac{6000 \text{ (reads)} + 4000 \text{ (writes)}}{10,000}$$

(master)

$$\text{Avg. Load} = \frac{6000 + (4 \times 4000)}{4 \times 10,000} = \frac{22,000}{40,000} = 55\%$$

(master + 3 slaves)

ORACLE

Let's take a look at the first common use for replication – scaling out reads.

When deciding how many slaves you need, it is important to understand that scaling out in this manner only scales out reads, not writes.

Each slave has to handle the same write load as the master.

The average load of the system can be described as:

Average Load = Sum of the Read Load PLUS Sum of the Write Load DIVIDED BY Sum of the Capacity of the server.

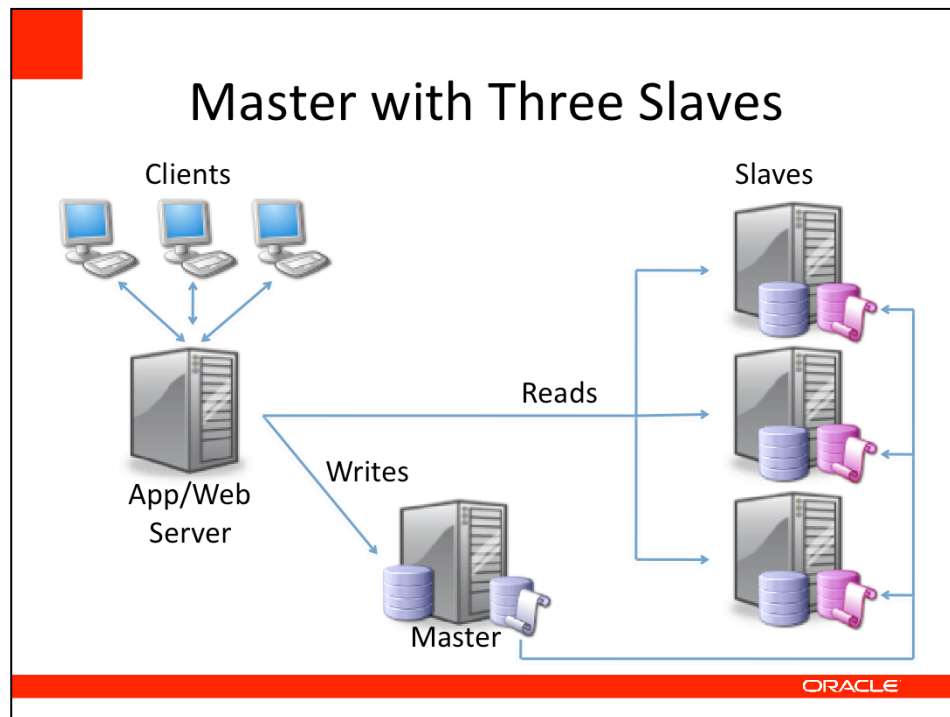
CLICK: So, let's assume that you have a single server with a total capacity of 10,000 transactions per second – with a read load of 6,000 transactions per second and a write load of 4,000 transactions per second.

You might think that if you add three slaves, your capacity would be at 25%, since you now have four servers.

It is quite common to forget that replication forwards to each slave all of the write queries that the master handles. So you cannot use this approach to scale writes, only reads.

CLICK: Once we add three slaves, each one will have to perform the 4,000 writes, while the 6,000 reads will be split among the three slaves.

So your actual capacity after adding three slaves is 55%.



Here is what your system will look like after adding three slaves.

For this example, we have our clients that are connecting to an application or web server. The server will send all of the writes to the master database, and all of the reads to the slaves.

At the same time, the slaves will be connecting to the master and replicating all of the reads from the master.

Scaling Out Writes, Not Reads

- Data sharding (splintering or partitioning)
- Reasons for sharding:
 - Placing geographically close to user
 - Reduce the size of the working set
 - Balancing the load
- Balancing and moving of shards
- Shard location is not fixed

ORACLE

We've looked at scaling reads by attaching slaves to a master and directing reads to the slaves while writes go to the master. As the load increases, it is easy to add more slaves to the master and to serve more read queries.

But, since all of the writes go to the master, if the number of writes increases enough, the master will become a bottleneck as the system scales up. We need a way to split the total amount of reads across multiple systems, and we accomplish this by data sharding – also called splintering or partitioning.

Sharding is separating the data into multiple “shards” – or separate databases – which are partitioned by primary keys from the database.

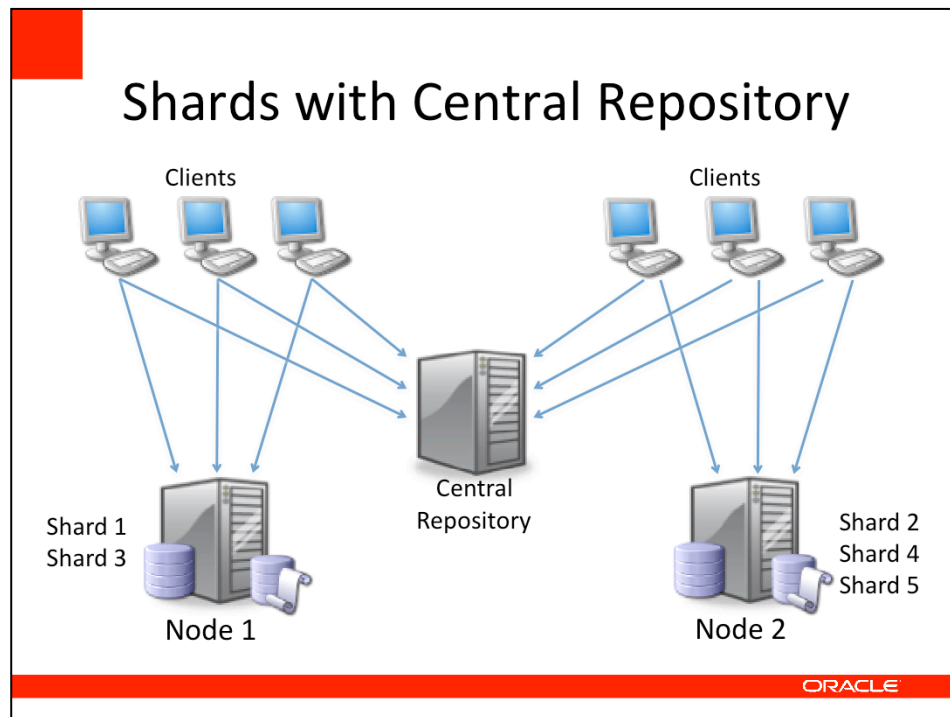
For example, you may decide to partition based upon the year from a customer order. Or you may partition the information by last name.

Reasons for sharding:

- placing data geographically close to the user helps to reduce latency – you could have databases spread out across the country.
- When you shard by month or year, you are reducing the size of the working set by searching through a smaller table which is more efficient than a larger table
- sharding makes it possible to balance the update load more efficiently – and if some shards too large, it is possible to split them into smaller shards

When sharding data, you can place several shards on a server, so when you rebalance the system, it is easy to move a shard to a different server, but repartitioning the data is very difficult.

With this architecture, the location of a shard is not fixed, and you need a method for translating a shard ID to the node where the shard is stored. This is typically handled with a central repository.



Here we have six clients, and since the location of a shard is not fixed, the central repository handles translating a shard ID to the node where the shard is stored.

Hot Standby

- Easiest of the replication topologies
- Hot standby duplicates master
- Potential problems:
 - Replication from new master, need to translate binlog positions from original master
 - Hot standby changes might not match slave
 - Repaired master might have additional binlog changes

ORACLE

The easiest of the replication scenarios for duplicating servers is the hot standby topology. It consists of a master and a dedicated server called a hot standby that duplicates the main master.

The hot standby server is connected to the master as a slave, and it reads & applies all of the changes as expected.

The idea is that when the main master fails, the hot standby provides a faithful replica of the master, and all of the clients and slaves can therefore be switched over to the hot standby and continue operating.

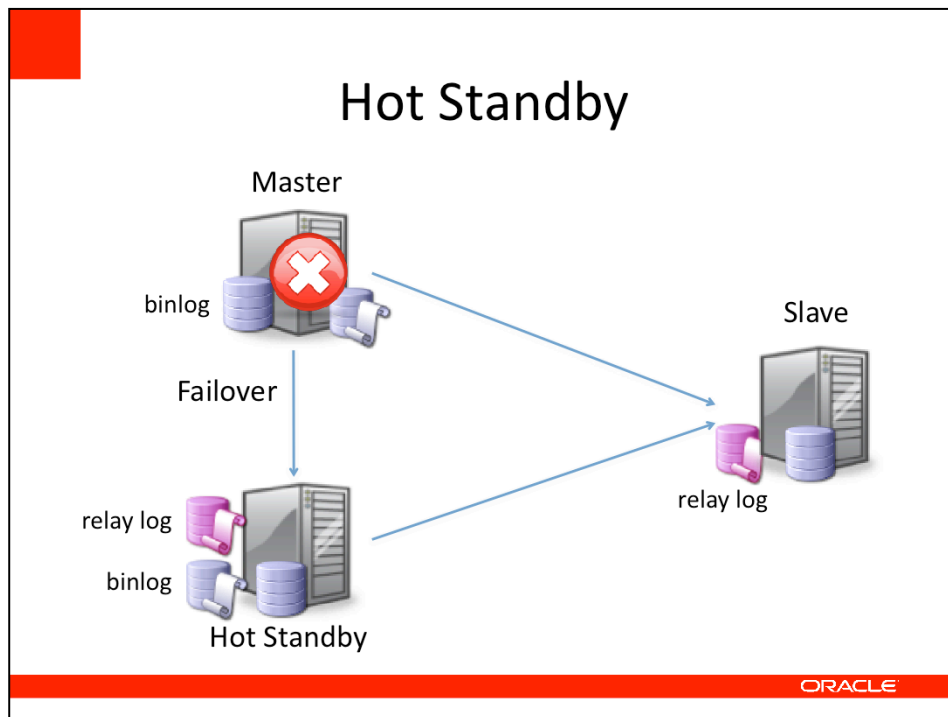
Failure of a master is inevitable – it isn't a question of IF the server will fail, but when. To ensure that operations will proceed, it is necessary to have a hot standby server available and to redirect all slaves to the hot standby when the main master fails.

This will give you a chance to check and see what happened with the main master, and maybe fix or replace it.

After you have repaired the main master, you have to bring it online and either set it to be the new hot standby, or redirect the slaves to the original master again.

Unfortunately, you have some potential problems:

- when failing over to the hot standby, you are replicating from a new master, so it will be necessary to translate the binlog positions from those of the original master to those of the hot standby
- when failing over a slave to a hot standby, the hot standby – which is the new master - might not actually have all of the changes that the slave has – the slave might have been updating data from the master faster than the hot standby
- when bringing the repaired master back into the configuration, the repaired master might have changes in the binary log that never left the server.



Here we have a master with one slave, and a hot standby slave.

The master is replicating to both the hot standby and the slave.

In addition to the relay log, the hot standby has a binlog, so when changes are made from the relay log, they are written to the binlog as well
– so when the hot standby takes over as the new master, it will continue to write changes to the binlog which will continue to be propagated to the slave.

The master fails – CLICK

You then failover to the hot standby - CLICK

And the slave now connects to the hot standby for updates.

Remote Replication

- Replication between two geographically separated data centers
- Disaster recovery
- Locate data closer to users
- Security/encryption via SSL
 - Use the built-in support to encrypt the replication
 - Use Stunnel to create SSL tunnel
 - Use ssh in tunnel mode

ORACLE

There are many reasons to replicate between two geographically separated data centers.

One reason is to ensure you can recover from a disaster such as an earthquake or a power outage.

You can also locate a site strategically close to some of your users in order to offer faster response times.

Even though you can use dedicated fiber, let's assume that you will use the Internet to connect the servers.

The events sent from the master to the slave should never be considered secure – in fact, it is easy to decode them to see the information that is replicated.

As long as you are behind a firewall and do not replicate over the Internet – for example, replicating between two data centers, this is probably secure enough..

As soon as you replicate to another data center in another town or on another continent, it is important to protect the information by encrypting it.

The details of generating, managing and using SSL certificates is beyond the scope of this presentation, but the ability to use SSL is available.

Using Replication for Backups

- Common replication backup uses
 - Backup and recovery
 - Point in time recovery (PITR)
- Stop slave, perform backup
 - Physical file copy (tar, WinZip)
 - Use mysqldump
- MySQL Enterprise Backup
 - No need to stop slave

ORACLE

There are two common backup strategies with MySQL replication – using replication to create a backup copy of the data and using backups taken previously for point in time recovery.

The easiest and most basic form of backup is a simple file copy. This does require you to stop the server for best results. Since you are using a slave for the backup, you can simply stop the slave and copy the data directory and any setup files on the server.

A simple method is to use the Unix tar command to create an archive. You can then move this archive to another system and restore the data directory. For Windows, you can use an archive program like WinZip.

You can also use the mysqldump utility. Mysqldump creates a set of SQL statements that re-create the databases and data when you rerun the statement.

The drawback to using mysqldump is that it takes a lot of time - a lot more time than the binary copies made by file-level or physical backups like MySQL Enterprise Backup or a simple offline file copy – and it requires a lot more storage space.

MySQL Enterprise Backup is much faster and more efficient than using mysqldump, as you can do a hot backup on the data without stopping the slave.

Report Generation

- Eliminates stress on the master server
- Stop slave during reporting
- Reports aren't as critical as business transactions
- Procedure may be automated
 - Stop replication on slave before midnight
 - After midnight, read last binlog position from master
 - Start the slave to run until this position is reached
 - Run reports
 - Restart slave

ORACLE

Creating reports from data on a server will degrade the server's performance – in some cases significantly. If you're running a lot of background jobs to generate reports, it is worth creating a slave just for this purpose.

You can get a snapshot of the database at a certain time by stopping replication on the slave and then running your reports or large queries without disturbing the master server.

For example, if you stop replication after the last transaction of the day, you can extract your daily reports while the rest of the business is continuing at its normal pace.

Reports aren't usually critical to the business as compared to processing normal transactions. It is better to setup a separate slave for reporting versus using a slave that might be part of your slave farm for scaling out reads.

And, you can easily automate the procedure for stopping the slave and running your reports. For example, let's pretend that we want to run a daily report. Here is what we need to do:

1. just before midnight, stop the replication on the slave so that no new events come in from the master
2. After midnight, check the binary log on the master and find that last event that was recorded before midnight. Obviously, if you do this before midnight, you might miss a few events for that day.
3. Record the binlog position of this event and start the slave to catch-up with the master until this position is reached.
4. When the slave has reached this position and stopped, you can run your reports.
5. Start the slave after reports have finished.

Binlog Filtering

- my.cnf file options: binlog-do-db, binlog-ignore-db
 - [mysqld]
 - binlog-ignore-db = two_db
- Example:
 - > USE two_db; INSERT INTO table1 VALUES (1),(2);
 - > USE two_db; INSERT INTO one_db.table1 VALUES (1),(2);
 - > USE two_db; INSERT INTO one_db.table2, three_db.table1 SET a = b;
- For example, instead of writing:
 - INSERT INTO books.pages values('Binlog','143');Instead, write:
 - USE books; INSERT INTO pages values ('Binlog','143');

ORACLE

It is possible to filter out statements from the binary log, so that changes to certain databases are not replicated in the slaves.

You might want to do this if you are using a slave for report generation, so that you only send over changes to the slave database that will be queried in a report.

To enable binlog filtering, you will need to edit your my.cnf file.

The option – binlog-do-db is used when you want to filter only statements belonging to a certain database

The option – binlog-ignore-db is used when you want to ignore certain databases but replicate all other databases

So, in the my.cnf file, you would need to add these options, along with the names of the databases you want to include or ignore. You can add multiple databases – just use one line for each database.

Consider what happens if the two_db database is filtered using the binlog-ignore-db.

- Line 1 changes a table in the current database two_db since it does not qualify the table name with a database name.
- Line 2 changes a table in a different database other than the current database.
- Line 3 changes two tables in two different databases – one_db and three_db – neither of which is the current database.

MySQL will filter on the active table – two_db – so none of these statements will be written to the binary log. To avoid these mistakes, issue a USE statement to make that database the current database.

For example, instead of writing: INSERT INTO books.chapters values ('MySQL','Chapter 1');

Write: USE books; INSERT INTO chapters values ('MySQL','Chapter 1');



Hierarchal Replication/Relay Slave

- A master is limited to number of replicated slaves
- Use a relay slave to lighten load on master
 - Use binlog to record changes from master
 - Use Blackhole engine to not write changes to database
- Relay slave introduces extra delay
- Hierarchal setup more difficult

ORACLE

Although a master server is quite good at handling a large number of slaves, there is a limit to how many slaves it can handle before the load becomes too high for comfort.

The total number of slaves that a master can handle depends upon the application, the size of the database, the load on the database, etc. Using a relay slave in this capacity is called hierarchal replication.

By default, the changes the slaves receives from the master are not written to the binlog of the slave, because there is no reason wasting disk space by recording these changes. If there is a problem with a slave, you can always recover by cloning the master or another slave.

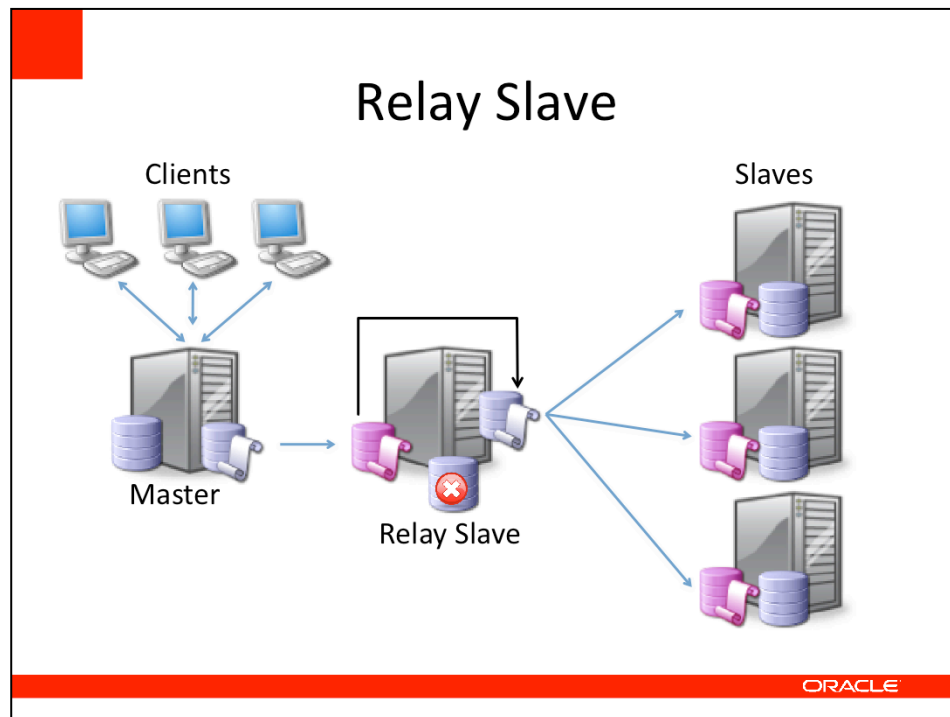
When using a relay slave, it is necessary to keep a binary log of all of the changes, because the relay slave needs to pass the changes off to other slaves. Unlike typical slaves, the relay slave doesn't actually need to apply the changes to a database of its own, because it doesn't answer queries.

A typical slave needs to apply changes to a DB, but not to a binary log. A relay slave needs to keep a binary log, but does not need to apply changes to a database.

The Blackhole database engine accepts all statements from the master and always reports success in executing them, but all changes are thrown away and not written to the database. So, the changes are written to the binlog, the other slaves will receive the change, but nothing is written to the database.

A relay slave introduces an extra delay in getting changes to the other slaves, which may cause the slaves to lag further behind the master.

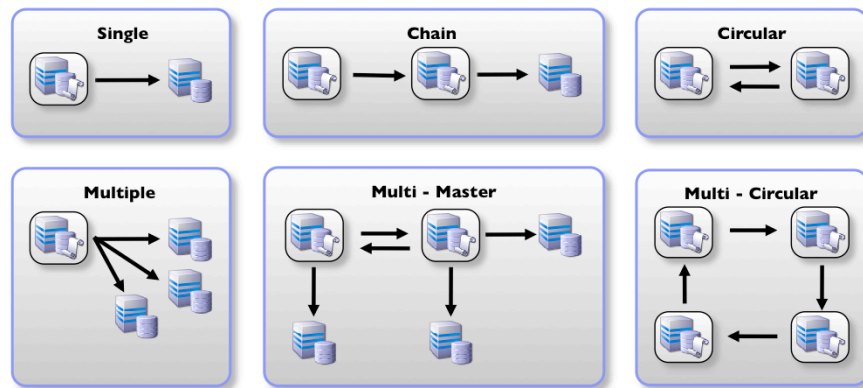
But this lag should be balanced against the benefits of removing some load from the master, since managing a hierarchal setup is more difficult than managing a simple setup.



Here we have a Master server that is sending all of the changes to a Relay Slave, which is then propagating the changes to the other slaves.

The relay slave doesn't write anything to its own database. It is simply taking the information from the master, putting it into a relay log, writing it to a binlog, and then the slaves are connecting to the Relay slave and updating their own databases.

Replication Topologies



ORACLE

Here are several different types of replication topologies. Each one has their own challenges in configuring and implementing them.

We have already looked at the single, and multiple setups, but the three on the right – the multi-master, circular and multi-circular are typically used where each server writes data specific to that server.

For example, in the circular and multi-master configuration, you might have data from the west coast populating the server on the left, and data from the east coast populating data on the right server. Each server is then replicating the data from the other server.

This would be the same for the multi-circular, where each server might have data written to it from a specific country or geographical region.

What's New in MySQL 5.6

- Crash-Safe Slaves
- Replication Checksums
- Reduced Binlog size – options for writing full/partial RBR (row-based replication) images
- Time-Delayed Replication
- Informational Log Events
- Remote Binlog Backups
- Server UUID's (Universally Unique Identifier)



ORACLE

Let's take a look at some of the new replication features in MySQL version 5.6.

Crash-Safe Slaves – 5.6 implements crash-safety for the slave by adding the ability of committing the replication information together with the transaction. This means that replication information will always be consistent with has been applied to the master database, even in the event of a server crash.

Replication Checksums - The replication event checksums are added to each event as it is written to the binary log and are used to check that nothing happened with the event on the way to the slave. Since the checksums are added to all events in the binary log on the master and transferred both over the network and written to the relay log on the slave, it is possible to track events corrupted because of hardware problems, network failures, and software bugs.

Reduced Binlog size – options for writing full/partial RBR images

Time-Delayed Replication – this allows you to setup a time delay when replicating to a slave, in case you accidentally do something like drop a table on the master, you can pause replication and restore data from a slave back to the master.

Informational Log Events - with row-based replication, you can activate a switch that will write the original statement into the binary log – versus just writing the row changes - and then you can see the original statement using SHOW BINLOG EVENTS or via the mysqlbinlog command.

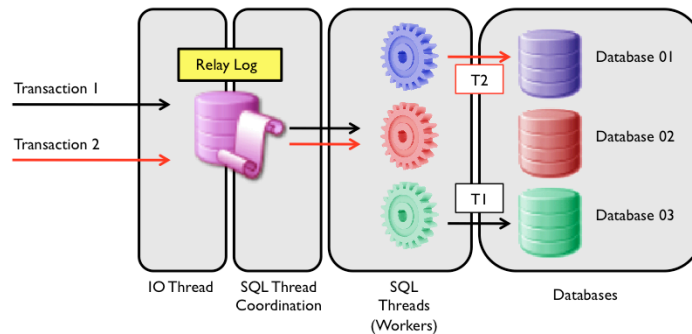
Remote Binlog Backups - By adding a flag, the binlog is written out to remote back-up servers, w/out having a MySQL database instance translating it into SQL statements, and w/out the DBA needing SSH access to each master server. (one use is to transfer many binlogs from many servers to one server to create a multi-master server)

Server UUID's - the server generates a true 128-bit value UUID in addition to the --server-id supplied by the user.

What's New in MySQL 5.6

Multi-Threaded Slave - Increased slave performance

Workload applied in parallel: Changes to each database are applied and committed independently



ORACLE

In 5.6 we also have the capability to have multi-threaded slaves.

At its core, MySQL replication is single-threaded. Events are pushed by the master to the slaves by the "dump thread".

At the slave, a reader (the "IO thread") reads each event in sequence and writes them to a local persistent queue, the "relay log".

Then a single threaded applier, the "SQL thread", reads and applies each event sequentially.

Contrary to the master, which executes transactions concurrently, the slave serializes the execution of each and every transaction.

But in 5.6, a multi-threaded slave would read transactions from the relay log and assign them to different worker threads, depending on the database the transaction is working on.

Transactions operating on the same database would then be guaranteed to be serialized.

For cross-database transactions, the slave waits until all preceding transactions that are working on the same database set have been completed.

